TASK 3

WEB APPLICATION SECURITY

LISHAJASMINE THANGAVEL

1. Executive Summary

This report documents the practical activities attempted for ApexPlanet Internship Task 3 — Web Application Security. The task focuses on identifying common web application weaknesses in a controlled environment, observing their impact, and documenting suitable mitigations.

A Damn Vulnerable Web Application (DVWA) instance was installed and configured locally in Kali Linux. The SQL Injection, Reflected Cross-Site Scripting (XSS), and Stored XSS demonstrations produced observable results. A CSRF test page was created and submitted, but the resulting password change was not independently verified. File Inclusion was not demonstrated because the installed PHP configuration reported that remote URL inclusion was disabled; the local-file test was skipped. Burp Suite was configured with a local proxy listener, but successful request interception and Intruder fuzzing were not confirmed. The HTTP security-header review and Apache hardening steps were not completed.

The report distinguishes completed demonstrations from attempted or incomplete activities. No claim is made that an unverified exploit, mitigation, or configuration change succeeded.

2. Aim and Objectives

Aim: To practise web application security testing against a deliberately vulnerable application and understand the impact and mitigation of common vulnerabilities.

- Install and configure DVWA in a local Kali Linux lab.
- Demonstrate SQL Injection and observe the effect of an injected query.
- Demonstrate Reflected and Stored Cross-Site Scripting using benign JavaScript alert payloads.
- Attempt a CSRF password-change scenario using a locally hosted proof-of-concept page.
- Review File Inclusion behaviour and document environment limitations.
- Configure Burp Suite as a local intercepting proxy and attempt request inspection.
- Review HTTP security headers and identify hardening measures to be applied.

3. Scope, Authorisation and Safety

All testing described in this report was performed against the user's own DVWA instance hosted locally in the Kali Linux virtual machine. The exercises were intentionally limited to a controlled lab. No public website, third-party system, or production account was targeted.

- DVWA is intentionally vulnerable and must remain isolated from untrusted networks.
- The VirtualBox lab used a Host-Only interface for local lab connectivity; NAT was used for package/internet access when required.
- Test credentials and payloads were demonstration-only. They must not be reused for real accounts.
- Screenshots and recordings should not expose real passwords, session cookies, API keys, or personal data.

4. Lab Environment and Tools

Component	Configuration / purpose
Host / virtualisation	Windows host with Oracle VirtualBox
Testing environment	Kali Linux virtual machine
Target application	Damn Vulnerable Web Application (DVWA), hosted locally
Web server / database	Apache and MariaDB
Runtime	PHP 8.4.27 (version shown by DVWA warning)
Browser	Firefox
Proxy tool	Burp Suite; local listener observed at 127.0.0.1:8080
DVWA security setting	Low (used for introductory vulnerability demonstrations)

DVWA was installed under the local web root and accessed at http://localhost/DVWA/. The setup page was used to initialise/reset the DVWA database, after which the application dashboard was accessible. Optional setup warnings were present; the PHP remote-include warning is discussed in Section 8.

5. Practical Activities and Results

5.1 SQL Injection

Objective: Demonstrate how untrusted input can alter the logic of a database query.

Procedure performed:

1. Opened the SQL Injection module in DVWA.
2. Entered the normal test value 1 to observe the baseline result.
3. Entered the test payload shown below and submitted it.

```
1' OR '1'='1' #
```

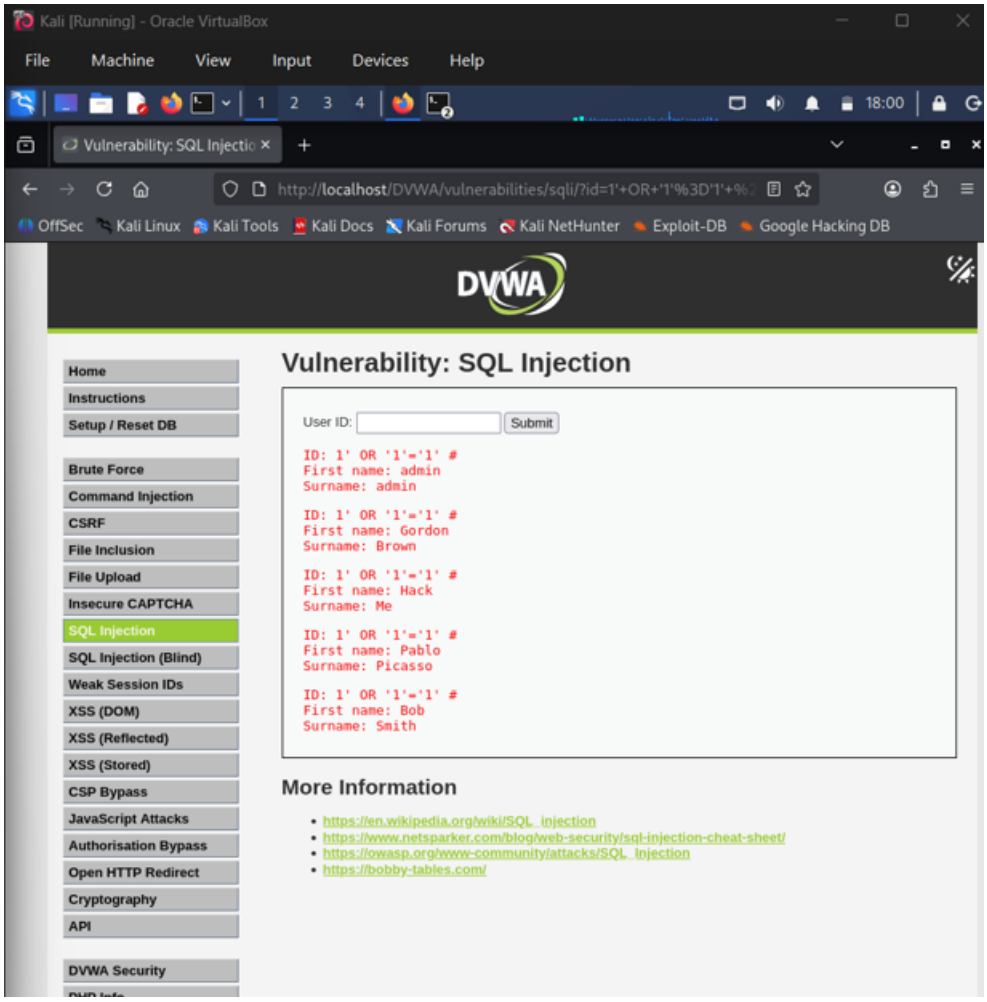
Observed result: The application returned multiple user records (five records were observed), rather than only the record associated with the normal input. The output included usernames and displayed name fields. This demonstrates that the supplied input affected the query logic. The practical did not establish that password hashes or plaintext passwords were extracted.

Security impact: An attacker may bypass intended query conditions, access unauthorised records, or modify data depending on the vulnerable query and database permissions.

Recommended mitigation:

- Use parameterised queries / prepared statements so input is treated as data, not SQL syntax.
- Validate input against the expected type, length, and allowed format.
- Apply least-privilege permissions to the database account.
- Avoid exposing detailed database errors to users.

Result: COMPLETED — observable SQL Injection behaviour was demonstrated.



5.2 Reflected Cross-Site Scripting (XSS)

Objective: Demonstrate execution of untrusted input reflected immediately in a browser response.

Procedure performed:

4. Opened the XSS (Reflected) module in DVWA.
5. Submitted a normal name value to observe the baseline greeting.
6. Submitted the benign JavaScript alert payload below.

```
<script>alert('XSS Demonstration')</script>
```

Observed result: The browser displayed an alert containing “XSS Demonstration”. This confirms that the application reflected the supplied input in a way that allowed script execution in the browser under the tested low-security configuration.

Recommended mitigation:

- Apply context-aware output encoding/escaping before inserting untrusted data into HTML.
- Validate input on the server; validation complements but does not replace output encoding.
- Deploy a restrictive Content Security Policy (CSP) as an additional defence-in-depth measure.
- Avoid unsafe DOM APIs and review client-side handling of untrusted content.

Result: COMPLETED — a reflected JavaScript alert was observed.

5.3 Stored Cross-Site Scripting (XSS)

Objective: Demonstrate a script payload being saved by the application and executed when the stored entry is displayed.

Procedure performed:

7. Opened the XSS (Stored) / guestbook module in DVWA.
8. Entered a demonstration name and the benign script payload below in the message field.
9. Submitted the guestbook entry.

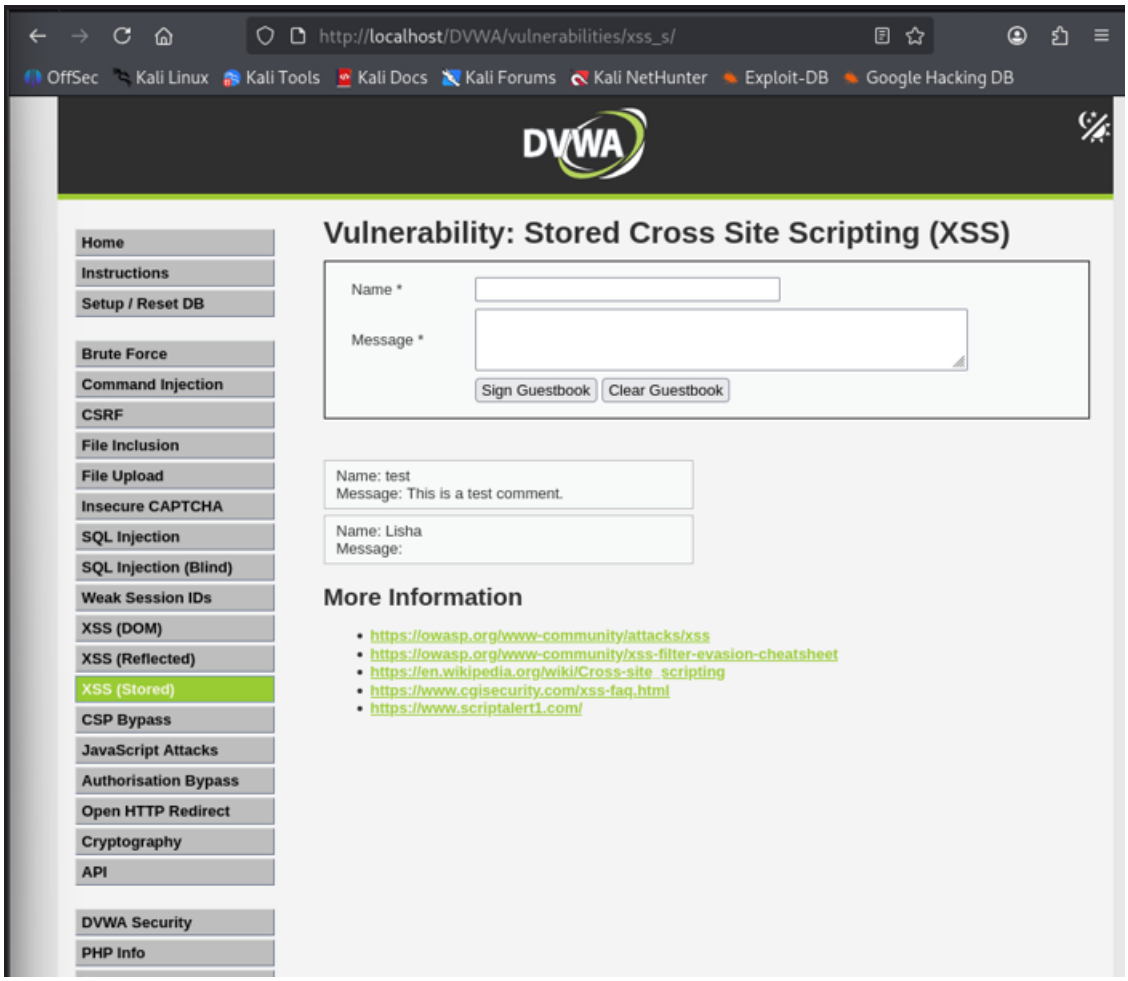
```
<script>alert('Stored XSS Demonstration')</script>
```

Observed result: A browser alert appeared after submission, and the submitted name and message were visible in the guestbook below. This is consistent with a stored XSS demonstration in which the application stores untrusted content and later renders it without safe output handling.

Recommended mitigation:

- Encode/escape untrusted content when it is rendered, including content retrieved from a database.
- Validate and constrain accepted input, while recognising that validation alone is not sufficient.
- Use a restrictive CSP and avoid unsafe script execution patterns.
- Review stored content and existing records when remediating a stored XSS issue.

Result: COMPLETED — the stored guestbook entry and alert were observed.



5.4 Cross-Site Request Forgery (CSRF)

Objective: Attempt a cross-site request scenario that changes a DVWA user's password.

Procedure attempted:

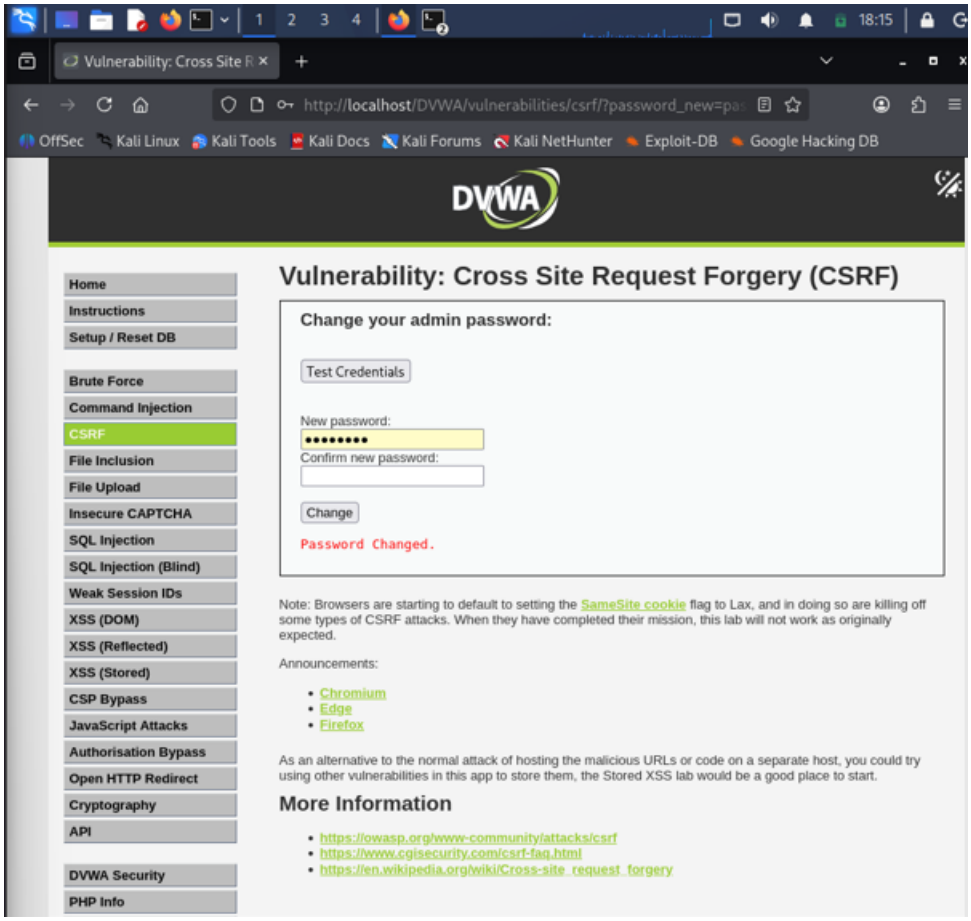
10. Opened DVWA's CSRF module, which provides a password-change form.
11. Submitted a demonstration password through the DVWA form; the application displayed a password-changed confirmation.

12. Created a local HTML proof-of-concept page containing a form targeting the local DVWA CSRF endpoint.
13. Hosted the page using Python's local HTTP server on port 8000 and opened it in Firefox.
14. Clicked the test button; the browser navigated back to the DVWA page.

Verification limitation: The browser navigation/redirect was observed, but the password change caused by the proof-of-concept request was not independently verified. Therefore, this report records the CSRF scenario as ATTEMPTED — NOT CONFIRMED. A redirect by itself is not proof that the forged request was accepted.

Recommended mitigation:

- Use unpredictable anti-CSRF tokens and validate them on every state-changing request.
- Use SameSite cookie attributes (Lax or Strict where appropriate) and Secure/HttpOnly cookie settings.
- Require appropriate re-authentication or confirmation for sensitive account changes.
- Do not perform state-changing operations through GET requests; use POST/PUT/DELETE with CSRF protection.
- Validate Origin/Referer headers as an additional defence where appropriate.



5.5 File Inclusion

Objective: Review the DVWA File Inclusion module and attempt to access a local file through the page parameter.

Observed environment limitation: The File Inclusion module displayed a warning that allow_url_include was not enabled. It also reported PHP version 8.4.27 and explained that the lab's remote-include behaviour depends on a PHP feature deprecated in newer PHP versions.

The local-file inclusion test was not carried out. Remote File Inclusion was therefore not demonstrated, and no sensitive file was read. This activity is recorded as SKIPPED / NOT DEMONSTRATED.

Recommended mitigation:

- Avoid passing user-controlled values directly to include/require functions.
- Use a strict server-side allowlist mapping short identifiers to known local templates.

- Disable remote file inclusion and keep PHP and application components supported and patched.
- Apply least-privilege file permissions and prevent access to sensitive system files.

5.6 Burp Suite Advanced

Objective: Intercept and modify login requests and perform fuzzing with Burp Suite Intruder.

Activities attempted:

15. Launched Burp Suite using a temporary project and Burp default configuration.
16. Verified that the Proxy listener was running at 127.0.0.1:8080.
17. Configured Firefox to use the local proxy and opened the local DVWA site.
18. Attempted to enable interception and generate a fresh request by refreshing DVWA.

Verification limitation: The listener configuration was confirmed, but a captured request was not verified in the available evidence. Intercepting and modifying a login request and using Intruder for fuzzing were not completed. This activity is recorded as PARTIALLY CONFIGURED / NOT FULLY DEMONSTRATED.

Recommended security practice: Use Burp only against systems for which testing is authorised. When performing a complete assessment, capture a request, inspect its method/headers/parameters, modify a harmless test parameter, forward the request, and document the response. Intruder should be configured with a small, controlled test wordlist and safe rate limits.

5.7 Web Security Headers

Objective: Analyse a test website using securityheaders.com and configure suitable HTTP security headers in Apache.

The browser Developer Tools route was attempted, but the response-header inspection was not completed. The securityheaders.com analysis and Apache configuration changes were also not completed. No header grade or successful hardening result is claimed. This activity is recorded as NOT COMPLETED.

Recommended headers and purpose:

Header	Purpose / example
Content-Security-Policy	Restricts permitted content sources and helps reduce XSS impact.
X-Content-Type-Options: nosniff	Prevents MIME-type sniffing.
X-Frame-Options: DENY	Helps prevent clickjacking by blocking framing (CSP frame-ancestors is the modern control).
Referrer-Policy: strict-origin-when-cross-origin	Limits referrer information sent to other origins.
Permissions-Policy	Restricts access to selected browser features.
Strict-Transport-Security	Enforces HTTPS after HTTPS is correctly configured; do not enable on an HTTP-only lab.

6. Consolidated Results

Task activity	Status	Evidence / notes
DVWA installation and setup	Completed	Local dashboard accessible; database initialised.
SQL Injection	Completed	Payload returned five user records.
Reflected XSS	Completed	Browser alert displayed.
Stored XSS	Completed	Guestbook entry displayed and alert appeared.
CSRF	Attempted — not confirmed	PoC page submitted; password change from forged request not verified.
File Inclusion	Skipped	PHP warning; local inclusion test not performed.
Burp Suite Advanced	Partially configured	Proxy listener at 127.0.0.1:8080; interception/Intruder not confirmed.
Web Security Headers	Not completed	No securityheaders.com result or Apache hardening evidence.

7. Overall Security Recommendations

- Use prepared statements for all database queries that contain user input.
- Encode untrusted data for its output context and apply a restrictive Content Security Policy.

- Protect all state-changing requests with CSRF tokens and appropriate cookie attributes.
- Avoid dynamic file inclusion based on user input; use fixed allowlists for templates.
- Keep PHP, Apache, DVWA dependencies, and the operating system updated.
- Use Burp Suite or another proxy to inspect application requests during authorised testing.
- Set and verify appropriate HTTP response security headers; test the final configuration after changes.
- Keep vulnerable training applications isolated and never expose them to the public internet.

8. Conclusion

Task 3 provided practical exposure to web application security testing using DVWA in a local lab. SQL Injection, Reflected XSS, and Stored XSS were successfully demonstrated through observable application/browser behaviour. A CSRF proof-of-concept workflow was created and submitted, but its security effect was not verified. File Inclusion, complete Burp Suite Advanced testing, and HTTP security-header analysis/hardening remain incomplete or were skipped because of environment and time limitations.

The completed demonstrations illustrate why applications must treat user input as untrusted, encode output safely, use prepared statements, and protect state-changing requests. The incomplete activities are explicitly identified so that the report reflects the work actually performed.